

NexusAI monitoring and runbook

Julian Calvo. AAI6850, Module 15 capstone. August 2026.

The contracts are live on Sepolia and Hoodi. Nothing is watching them yet. This is what I'd stand up before a first mainnet listing and what I'd do when each alert fires.

1. What gets watched

I would watch three things.

Safety, meaning anything that says the protocol is enforcing less than it should: a tripped circuit breaker, a stale oracle, a pause, a privileged role change.

Money. Escrow that's been sitting past its window, dispute rate, slashing, how much of the emission budget has been used. Most of this is slow-moving and a daily check would catch it, but slashing and unexplained escrow movement are worth an alert.

Liveness. Cross-chain messages that were sent and never executed, RPC health, whether the reporters are still submitting.

Severity tiers:

Tier	Meaning	Response
SEV1	Funds or authority at risk	Page immediately
SEV2	Protocol enforcing less than designed	Page during business hours
SEV3	Degraded, self-correcting	Ticket

Pipeline: contract events go to a subgraph, a poller reads derived state that isn't in events (escrow age, timelock queue), both feed Prometheus, alerts go to PagerDuty.

2. Signals that matter most

Oracle staleness

`ReportSubmitted` gives the last report time per model. If the newest report for an active listing is older than `stalenessWindow` (1 day), purchases are already reverting and nobody has noticed.

Alert at 75% of the window. SEV2.

CircuitBroken

`CircuitBroken(modelId, previousAccuracyBps, newAccuracyBps)`. Sales for that model stop until an admin clears it. This is the system working, but it needs a human to look at whether the model actually degraded or a reporter is wrong.

Alert on any occurrence. SEV2. Two in a week for the same model is SEV1, because either the model is unstable or a reporter is.

DisputeOpened rate

Baseline should be near zero. A spike either means a provider's model broke or someone found a way to farm refunds.

Alert if disputes exceed 5% of purchases over a rolling 7 days. SEV2.

Slashed

Slashed(account, amount, bps, reason) only happens through the Timelock, so it should always be expected. An unexpected one means a role is somewhere it shouldn't be.

Alert on any occurrence. SEV1 if no corresponding timelock execution exists.

Cross-chain send vs execute lag

Count MessageSent on the source against MessageConsumed on the destination. A growing gap means the relayer is down or messages are being dropped.

Alert if any message is unconsumed after 30 minutes. SEV2. After 4 hours, SEV1.

Polled state

Three things aren't in events, so a poller has to go look. Escrow age: how long each open purchase has sat past its dispute window without releaseEscrow. Providers not claiming means either they stopped paying attention or something reverts, and 3 days past window is worth a ticket (SEV3). Emission ceiling usage: mintedThisEpoch against emissionCeiling, alert at 90% because minting is about to start failing (SEV3). Unexecuted timelock operations: anything eligible and still unexecuted 48 hours past its delay, which is either forgotten or somebody scheduled something they'd rather nobody executed (SEV2).

Pause state

Any Paused or Unpaused on the marketplace or vault. Should always match a known incident.

Alert on any occurrence. SEV1 if unexplained.

Privileged role changes

RoleGranted and RoleRevoked on every contract. After handoff, the only legal source of these is a timelock execution.

Alert on any occurrence with no matching timelock operation. SEV1.

3. Event to metric map

Contract	Event	Metric
AIModelMarketplace	ModelRegistered	listings created
	LicencePurchased	sales, GMV, fee, burn
	DisputeOpened	dispute rate
	DisputeResolved	outcome split
	EscrowReleased	escrow drain
	ModelDelisted	delistings, with reason
	ExclusiveLicenceIssued	exclusive issuance
AIPerformanceOracle	ReportSubmitted	reporter uptime, last-report age
	AggregateUpdated	current median per model
	CircuitBroken / CircuitCleared	breaker state
	ReporterAdded / ReporterRemoved	committee size
StakingVault	Staked / Unstaked	TVL, tier mix
	RewardFunded / RewardPaid	reward runway
	Slashed	slash events
	EmergencyWithdrawn	forfeited rewards
NexusAIToken	EpochRolled	epoch boundary
	EmissionCeilingUpdated	ceiling changes
ComplianceRegistry	Attested / Revoked	attestation churn
	JurisdictionBlocked / Denied	blocklist changes
SealedBidLicenceAuction	AuctionCreated / BidCommitted / BidRevealed	funnel and reveal rate
	AuctionSettled	clearing price
	ExclusiveLicenceIssuanceFailed	settle succeeded but licence didn't issue
CrossChainRegistry	ModelPublished / LicencePublished	outbound
	MessageConsumed	inbound, and the lag calc
	TrustedRemoteSet / RouterUpdated	trust boundary changes
Governance	ProposalCreated / VoteCast / ProposalExecuted	governance activity

settle wraps the licence issuance in a try/catch so a failed issuance doesn't strand an entire auction. That means the failure is silent unless someone is watching for this event. Alert on it. SEV2.

4. Dashboards

I would build four panels.

1. **Protocol health.** Sales, GMV, fee, burn, active listings, breaker state per model.
2. **Oracle.** Median per model, last report age, reporter submission counts, breaker history.
3. **Cross-chain.** Sent vs consumed per lane, oldest unconsumed message, nonce high-water mark per chain.
4. **Governance and roles.** Open proposals, queued timelock ops, current role holders per contract compared against expected.

5. Runbook

Common first steps for anything: check whether the marketplace or vault is paused, check whether a timelock execution explains it, check the block explorer for the transaction, and write down what you found before you change anything.

Unexplained privileged change. SEV1. Pause the marketplace and vault. Get the full role list from `governance-handoff.js --dry-run`. Compare against the role matrix in the technical doc. If a key is compromised, the Timelock can revoke, but it takes 2 days, so pause first.

Circuit broken. Look at the individual reports behind the median. If the reports agree, the model degraded and the provider should be told. If one reporter is an outlier, investigate that reporter before clearing.

Oracle stale. Find out which reporters stopped. If it's the pipeline, restart it. If it's one reporter, the quorum should have absorbed it, so check why it didn't.

Elevated disputes. Pull the reasons off `DisputeOpened`. If they cluster on one model, delist it pending investigation. If they're spread, the mechanism is more likely at fault than the providers.

Slash with no timelock op. SEV1. Treat as a compromised key. Pause, then work the role inventory.

Cross-chain stall. Check the relayer process and the destination RPC. The messages aren't lost, they're unconsumed, and nonce checks mean replaying them later is safe. If the router itself is suspect, `setTrustedRemote` to zero on the destination stops inbound until it's sorted.

Emission ceiling pressure. Either the schedule is wrong or something is minting more than expected. Governance can raise the ceiling up to `MAX_EPOCH_EMISSION`, but not past it, and that's a 2-day proposal.

Unexecuted timelock op. Read the `calldata`. If it's legitimate and forgotten, execute it. If nobody can say who scheduled it, cancel it.

After any SEV1 or SEV2: write it up, add a test if the cause was code, and add an alert if the cause was something nothing was watching.

6. Routines

Daily: reporter submission counts, unconsumed cross-chain messages, open disputes. Weekly: role inventory against the expected matrix, escrow age, reward runway. Monthly: re-run the ML evaluation, compare drift, review accepted Slither findings against the current code.

7. Gaps in this design

- No monitoring for a model that is technically passing its floor but has quietly drifted just above it. Drift is in the oracle payload but nothing alerts on it yet.
- No way to detect a reporter submitting plausible but fabricated numbers. The median helps against a minority, monitoring doesn't help at all.
- Escrow age needs a poller because it isn't in an event, which means it fails silently if the poller dies. It should have its own heartbeat.